

IA & Craft

À la croisée des chemins

Pourquoi un logiciel ne se "prompte" pas, il se construit avec un agent crafté.

LA CONSTITUTION VIVANTE

AGENTS.MD
SKILL.MD
TECH-DEBT

L'Artisan Traditionnel (Tink)



- **Tink (L'Artisan)** : Intelligence humaine manipulant directement la matière et les ressources.
- **Craftsmanship** : Les 4 piliers :
 - **Logiciel bien conçu** : La qualité comme fondation.
 - **Ajout constant de valeur** : Chaque geste compte.
 - **Communauté de professionnels** : Le partage de pratique.
 - **Partenariat productif** : Entre le métier et la technique.

LA CONSTITUTION VIVANTE

Le mirage du prompt magique

- **L'illusion du prompt unique** : Croire qu'une instruction brillante suffit à garantir un comportement stable.
- **L'instabilité de l'agent seul** : Sans cadre, l'Agent devient amnésique ou incohérent d'une session à l'autre voir au sein même d'une fenêtre contextuelle.
- **La rupture** : Passer de l'instruction ponctuelle au **système de travail**.
- **Le constat** : Un bon agent a besoin d'un contexte durable, pas seulement d'intelligence.

LA CONSTITUTION VIVANTE

The background of the slide is a detailed illustration of a futuristic workshop. On the left, a statue of a woman is constructed from circuitry and glowing blue lines, standing on a stone pedestal inscribed with 'LA CONSTITUTION VIVANTE'. To her left is a workbench with various tools. On the right, a stack of three books is labeled 'AGENTS.MD', 'SKILL.MD', and 'TECH-DEBT'. In the background, a computer monitor displays a hexagonal diagram and the text 'LE CORPCE NÉMOIRE'. The floor is made of stone tiles with glowing circuit patterns, and the ceiling is dark with a network of glowing nodes and lines.

Le mirage du prompt magique

un agent qui ne se trace pas devient un agent qui ne s'audite pas. Et un agent qui ne s'audite pas finit par être désactivé, souvent après un incident...

Avant / Après — Ce qui change vraiment

Prompt seul

- **Mémoire volatile** : l'agent oublie ce qui ne tient plus dans la fenêtre.
- **Décisions implicites** : architecture, Git, tests et review restent dans la tête de l'humain.
- **Qualité instable** : une bonne réponse peut être suivie d'une dérive dès la session suivante.
- **Audit faible** : difficile d'expliquer après coup pourquoi une décision a été prise.

Système de travail

- **Mémoire versionnée** : AGENTS.md , ADRs, guides, skills, backlog, tech-debt.
- **Décisions explicites** : l'agent hérite d'un cadre, pas seulement d'une demande.
- **Qualité reproductible** : mêmes garde-fous, même protocole, même exigence.
- **Audit réel** : on peut relire les règles, les écarts et les corrections.

La mémoire du projet n'est pas le contexte de la session

- Le **contexte de session** est court, fragile et partiellement invisible.
- La **mémoire projet** est persistante, relue, versionnée et partagée.
- C'est elle qui permet à l'agent de repartir correctement **même plusieurs jours plus tard**.

Ce que la session sait

- La demande du moment
- Quelques fichiers récents
- Un historique local et incomplet

Ce que le projet sait

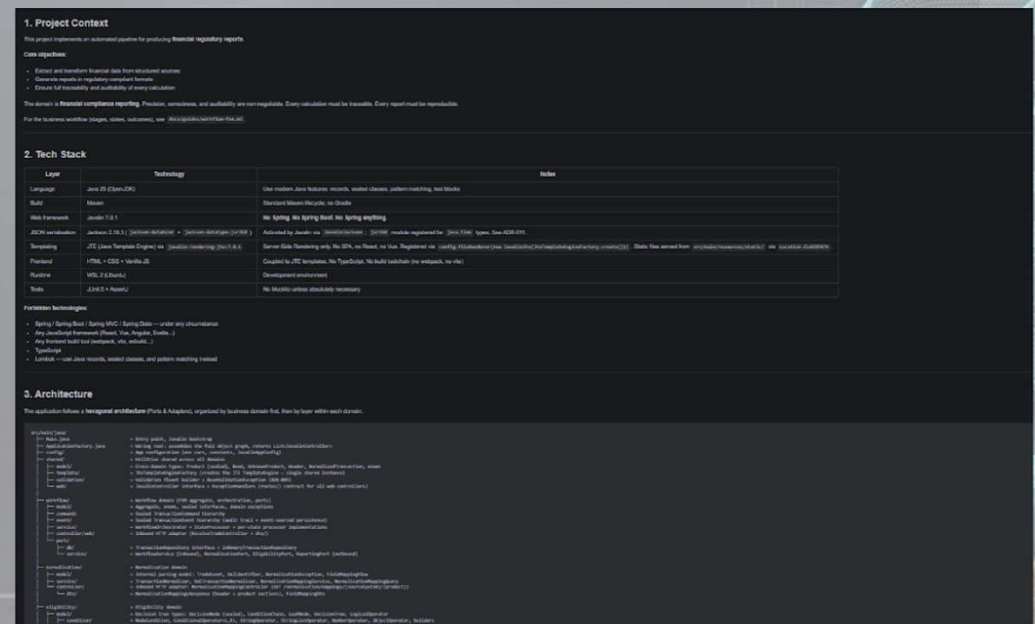
- Les décisions d'architecture
- Les pièges déjà rencontrés
- Les procédures récurrentes
- L'état du backlog et de la dette

Pygmalion sculpte Galathée : La Constitution

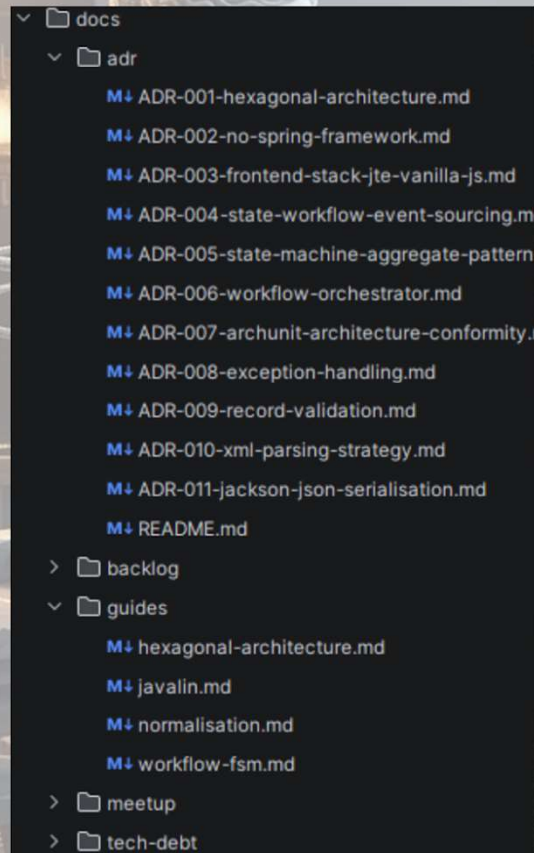


La Constitution — La Loi (AGENTS.md)

- **Le point d'entrée unique** : L'agent a l'obligation de lire ce fichier avant toute action.
- **Contexte Métier** : Reporting financier réglementaire (Précision et Auditabilité non-négociables).
- **Stack "No Magic"** : Java 25 pur, Javalin, pas de Spring, pas de Lombok. On garde la maîtrise totale.
- **Architecture Hexagonale** : Le design est explicite et visuel directement dans la "loi" du projet.



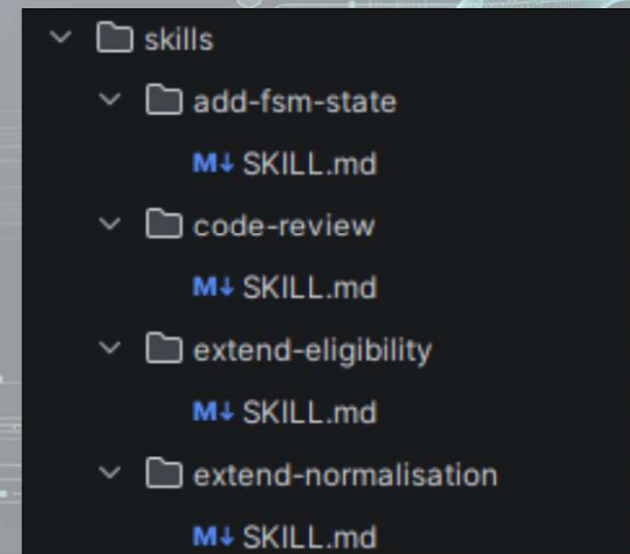
La Constitution — La Mémoire (Docs & ADRs)



- **11 ADRs (Architectural Decision Records)** : Chaque choix structurant est gravé dans le marbre et versionné.
- **Guides de Domaine** : Hexagonal, Javalin, Normalisation. On ne surcharge pas le prompt, on route vers le savoir.
- **Backlog & Tech-debt** : Une mémoire opérationnelle qui évite à l'agent de naviguer à vue.

La Constitution — L'Exécution (Skills)

- **Encapsuler le geste expert** : On extrait les gestes répétitifs et complexes dans des `SKILL.md` dédiés.
- **Procédures expertes** :
 - `add-fsm-state`
 - `code-review`
 - `extend-normalisation`
- **Le résultat** : L'agent ne "devine" plus comment étendre le système, il suit un manuel de vol spécifique au projet.



Pygmalion sculpte Galathée : La Forteresse technologique



Forteresse Architecturale — La Structure Imposée

- **Architecture Hexagonale** : Une structure de dossiers et de dépendances imposée dès la racine du projet.
- **Bird-eye View** : L'agent dispose d'une vision claire du "Pattern général" directement dans ses instructions.
- **Cœur de Domaine** : `model` et `service` forment le noyau, sans aucune dépendance vers les adaptateurs (controllers ou ports).
- **Injection de Dépendances** : Le câblage des implémentations appartient exclusivement à `ApplicationFactory`.

```
src/main/
├── domain/
│   ├── model/
│   │   ├── entity/
│   │   ├── value/
│   │   └── event/
│   ├── service/
│   │   ├── usecase/
│   │   └── orchestration/
│   ├── controller/
│   │   ├── web/
│   │   ├── dtu/
│   │   ├── kafka/
│   │   ├── db/
│   │   ├── web/
│   │   └── messaging/
│   └── port/
│       ├── http/
│       ├── kafka/
│       ├── db/
│       └── messaging/
├── src/main/java/
│   ├── org/
│   │   ├── example/
│   │   └── application/
│   └── test/
│       ├── org/
│       │   ├── example/
│       │   └── application/
│       └── test/
└── src/test/java/
```

General pattern for any domain

- Domain model: entities, value objects, domain events (Java records and sealed classes - no DTOs here)
- Business logic, use cases, orchestration
- Inbound adapters (this, no business logic)
- JavaSE HTTP handlers (only if multiple adapter types)
- Request/response records for this controller
- Kafka consumers (if applicable)
- Outbound adapters and inbound port definitions
- JDBC repositories, SQL queries
- Outbound HTTP clients (if applicable)
- Message producers (if applicable)

JTE templates

- Base layouts
- Page templates (one subfolder per domain)

Mirrors src/main/java structure exactly

Hexagonal architecture rules:

- `model` and `service` form the domain core — no dependencies on controllers or ports
- `controller` packages are inbound adapters: depend on `service`, never on port directly
- `port` packages are outbound adapters: depend on `model`, never on `controller`
- Services depend on port interfaces only — never on concrete port classes
- DTOs live in `controller` (inbound) or `port` (outbound) — never in `model`
- No circular dependencies between domains; shared concepts go in `shared/`
- Wiring of concrete implementations belongs exclusively to `ApplicationFactory` — services must never instantiate their own collaborators; receive them via constructor injection
- Before creating a type in `domain/model/`, ask: will any port interface in `workflow/port/service/` or any other domain reference it? If yes — `shared/model/`. See the placement checklist in [docs/guides/hexagonal-architecture.md](#) §4.

For placement rules, dependency matrix, and known pitfalls, see [docs/guides/hexagonal-architecture.md](#).

Forteresse Architecturale — Le Guidage Précis

Guide: Hexagonal Architecture

Read this guide when creating a new class, a new domain, or a new adapter. For architectural rationale, see: ADR-001.

1. Dependency rules

```
controller → service → port (interface)
               ↑
               port (implementation)
```

From	May depend on	Must NOT depend on
model	nothing	service, controller, port
service	model, port interfaces	controller, concrete port classes
controller	service, model	port directly
port (impl)	model	controller, service

No circular dependencies between domains. Shared concepts go in `shared/`.

2. Where to place a new class

What you are creating	Where it goes
Domain entity, value object, enum	<code><domain>/model/</code>
Business use case / orchestration	<code><domain>/service/</code>
HTTP handler (Javalin)	<code><domain>/controller/</code> (or <code><controller>/web/</code> if multiple adapter types)
Outbound port interface	<code><domain>/service/</code> or <code><domain>/model/</code>
Outbound port implementation (DB, HTTP)	<code><domain>/port/db/</code> or <code><domain>/port/web/</code>
Cross-domain concept	<code>shared/</code>
DTO for inbound HTTP	<code><domain>/controller/</code>
DTO for outbound call	<code><domain>/port/</code>

- **Le Guide comme Boussole** : Pour éviter de surcharger le prompt, les détails fins sont déportés dans des guides spécialisés.
- **Matrice de Dépendances** : Des règles explicites sur ce qui est autorisé ("May depend on") et strictement interdit ("Must NOT depend on").
- **Règles de Placement** : Un tableau de routage précis indique à l'agent où créer chaque nouvelle classe selon sa nature.

Forteresse Architecturale — Fail-Fast & Pitfalls

- **ArchUnit (Fail-Fast)** : 28 règles automatisées bloquent toute dérive architecturale dès le build.
- **Contrôle Hybride** : On ne repose ni uniquement sur l'agent, ni uniquement sur l'humain pour la conformité.
- **Anticiper les pièges (Pitfalls)** : Chaque erreur rencontrée est consignée pour ne "prendre le mur" qu'une seule fois.

6. ArchUnit test coverage

The following constraints are enforced automatically. Tests live in `src/test/java/com/regulatoryreporting/architecture/`

Test class	Rules enforced
HexagonArchitectureTest	<code>model</code> must not depend on <code>service</code> , <code>controller</code> , or <code>port</code> ; <code>service</code> must not depend on <code>controller</code> ; <code>controller</code> and <code>port</code> must not depend on each other
HexagonArchitectureTest	inter-domain isolation: <code>normalization</code> , <code>eligibility</code> , <code>reporting</code> , and <code>shared</code> must not import from each other's domain packages (3 directional rules)
HexagonArchitectureTest	<code>domain/model</code> types must not be imported outside their own domain — enforced via <code>normalizationmode</code> , <code>sharedmode</code> , <code>portmode</code> , <code>sharednormalization</code> , <code>eligibilitymode</code> , <code>sharedreporting</code> , and <code>sharedshared</code>
AggregateContextTest	Only <code>workflow.service</code> may call <code>TransactionAggregate.Apply()</code> and <code>TransactionAggregate.Fail()</code>
ExceptionsHandlingTest	Caught exceptions must be wrapped-rethrown or logged — silent swallowing is forbidden

When to add a new test: see AGENTS.md §5 and §7 rule 13

7. Known pitfalls

- Business logic in a controller – controller must delegate immediately to a service. Validation, transformation, and decisions belong in `service` or `model`.
- Concrete port class referenced from a service – the service must only reference the port interface. If you find `new SomeDependency()` inside a service, it is a violation.
- Domain object used as a DTO – records in `model` must not be serialized directly to HTTP responses or used as database row mappings. Create a separate DTO in `controller` or `port`.
- Circular domain dependency – domain A importing from domain B, domain B exports the shared concept to `shared`.
- Crab package created prematurely – do not create `controller/web` if there is only one type of inbound adapter. Add the sub-package only when a second adapter type appears.

Architecture conformity

Architectural constraints are enforced automatically via ArchUnit (ADR-007 — implemented). ArchUnit tests live in `src/test/java` under the `architecture` package and run as part of the standard Maven test phase. They must never be disabled.

A new ArchUnit test must be added in the same commit when any of the following occurs:

- A new domain package is introduced (add the full set of inter-domain isolation rules for it)
- A new architectural pattern is adopted that must not be violated (e.g., a new aggregate, a new port interface, a new access restriction)
- A constraint is identified that, if violated silently, would cause a costly refactoring later

Pitfall: `allowEmptyShould(true)` silences rules that match no classes — remove it from any rule as soon as its target package is stabilised. Exception: negative rules (e.g. `newMethods().should()`) that enforce absence of violations must keep `allowEmptyShould(true)` permanently, because finding zero matches is the expected healthy state. Never add it to positive rules (`classes().should()`).

ArchUnit rule catalogue (28 rules across 4 test classes):

- `HttpContextAccessor` – layer dependency on inter-domain identity, model encapsulation per domain, and `ControllerMethodParameterTypes` (no `Model` type as a method return type in any `Controller` class)
- `WebControllerConventionTests` – controller naming and identity conventions, and `RestControllerMustNotBeAWebController` (every `RestController` – taking method in a non-`WebController` controller must carry `ApiController`)
- `ExceptionHandlerInjectionsTest` – exception handling policies (ADR-006)
- `AggregateAccessTest` – aggregate access restrictions

Cas concret — La règle devient un test

- **Point de depart** : je fais une demande d'evolution qui pousse vers une mauvaise dependance entre couches.
- **Premier garde-fou** : l'agent detecte que la solution viole l'architecture hexagonale.
- **Iteration utile** : on retravaille ensemble la proposition jusqu'a trouver le bon placement.
- **Capitalisation** : l'agent propose ensuite d'ajouter une regle ArchUnit pour interdire a l'avenir cette derive, venant de moi comme de lui.

Avant

Une regle connue seulement par les personnes presentes dans la discussion.

Apres

Une contrainte executable que le build peut faire respecter tout seul.

Garde-fous — La Stack "No Magic"

- **Maîtrise Totale** : Java 25, Maven, et Javalin pour éviter la "magie" des frameworks trop lourds.
- **Zéro Framework Front** : JTE et Vanilla JS. Pas de React, pas de build toolchain complexe.
- **Technologies Proscrites** : Liste explicite d'interdictions (Spring, Lombok, TS) pour forcer l'usage des fonctionnalités natives.
- **L'Agent sous Contrainte** : Il ne peut pas "improviser" une dépendance magique pour résoudre un problème.
- **Effet de style** : Les interdits techniques ne bloquent pas seulement des libs ; ils redirigent la forme même des solutions produites.

2. Tech Stack

Layer	Technology	Notes
Language	Java 25 (OpenJDK)	Use modern Java features: records, sealed classes, pattern matching, text blocks
Build	Maven	Standard Maven lifecycle; no Gradle
Web framework	Javalin 7.0.1	No Spring, No Spring Boot, No Spring anything.
JSON serialisation	Jackson 2.18.3 (<code>jackson-databind</code> + <code>jackson-dataformat-jrjse</code>)	Activated by Javalin via <code>JavalinJackson</code> ; <code>Jrjse</code> module registered for <code>java.time</code> types. See ADR-011.
Templating	JTE (Java Template Engine) via <code>Javalin-rendering-jte</code> (7.0.3)	Server-Side Rendering only. No SPA, no React, no Vue. Registered via <code>config.fileRenderer(new JavalinJteTemplateEngineFactory.create())</code> . Static files served from <code>src/main/resources/static/</code> via <code>Location.CLASSPATH</code> .
Frontend	HTML + CSS + Vanilla JS	Coupled to JTE templates. No TypeScript. No build toolchain (no webpack, no vite)
Runtime	WSL 2 (Ubuntu)	Development environment
Tests	JUnit 5 + AssertJ	No Mockito unless absolutely necessary

Forbidden technologies:

- Spring / Spring Boot / Spring MVC / Spring Data — under any circumstance
- Any JavaScript framework (React, Vue, Angular, Svelte...)
- Any frontend build tool (webpack, vite, esbuild...)
- TypeScript
- Lombok — use Java records, sealed classes, and pattern matching instead

Garde-fous — Le Code Style comme Loi

- **Immuabilité par défaut** : Usage intensif de `records` et de champs `final` .
- **Sémantique Moderne** : Pattern matching et expressions `switch` préférés aux chaînes `if/else` .
- **Zéro Null** : Interdiction du `null` en retour au profit d' `Optional` .
- **Validation aux Frontières** : Validation systématique via constructeurs compacts dans les records.
- **Typage Explicite** : Interdiction du mot-clé `var` pour maintenir une clarté absolue pour l'humain et l'agent.

Java style

- Prefer immutability: use `records` for data objects, `final` fields and unmodifiable collections. Do not add `final` to class declarations — this is a design decision made case by case, not a default.
- Prefer `sealed` interfaces/classes for domain types with a fixed set of variants
- Use pattern matching (switch expressions, instanceof patterns) over chains of `if/else`
- Use text blocks for multi-line strings (SQL, HTML, snippets, JSON)
- Avoid `try` or `return` expressions when a value may be absent; never return `null` from a public method. `Optional` is a return type only — never use it as a method parameter or as a record/class field. The sole exception: `Optional` fields are permitted on aggregates to make absence explicit at the domain level — never replace them with `null` and a comment.
- No raw types, generics must always be fully typed
- Always use explicit `import` statements — never use fully qualified class names within code (e.g. `java.util.Optional.ifPresent` is forbidden; add `import java.util.Optional;` instead). This rule is not enforced automatically — ArchUnit cannot distinguish FQN-refs from a properly imported class at bytecode level. Compliance is enforced during code review only.
- No checked exceptions in domain logic: wrap them in unchecked domain-specific exceptions
- On `try` or `return` expressions and everywhere else, parentheses are only present when building a constrained DSL, either by:
 - reversing for grouped related inputs (extract them from their own file instead). The canonical pattern for constrained DSL builders in this project is a `Builder` class (sometimes, or by type that owns the builder) declares `public interface <T> T` as first type to enforce step-by-step construction order at compile time (e.g. `FileIdemp <? FileIdemp> <? operator >`). The ternary step declares a concrete domain type (e.g., `void condition()`). This pattern is preferred over chained `with()` methods and over a single constructor with many parameters. `StringConditionalizer`, `AspectConditionalizer`, and `ConditionChain` are the canonical examples. This rule applies to both production code and test code. ArchUnit does not enforce it in test classes — manual review is the only safeguard.
- No comments in production code — code must be self-explanatory
- No nested ternary expressions — avoid nesting ternaries to ensure code readability
- Never use `var` — always declare explicit types for all local variables
- Use `__` for unused pattern binding variables (e.g., `case ReportFailure() __`) in switch expressions
- Enums carry behavior: when a fixed set of values has associated state or logic, make the enum carry it directly (fields, abstract methods, or interface implementations) rather than externalizing the logic into a `switch` elsewhere. `TransactionState` (which carries `cancelCommand`, `terminate`, and the list of accepted command classes) is the canonical example of this pattern.
- Record validation via compact constructor: every record whose fields originate from user input, parsed data, or inter-domain transfer must validate its fields in a compact constructor using `validation.begin()` from `shared/validation/`. Use `withName()` for string fields, `guardNotNull()` for object fields and `guardNotNull(isOptional)` for a long when a non-empty collection is required. `AccountValidationException` is thrown automatically on the first violated constraint — do not catch it inside the record. The canonical example is `AccountValidationException`.
- Threading safety in multi-threaded web applications: every object held as a field in a long-lived class (service, controller, port) must be either (a) immutable (record), (b) thread-safe, unmodifiable collection, or (c) a library type explicitly documented as thread-safe. If a candidate object is mutable or has unknown thread safety, flag it by Pygmalion before introducing it. Do not assume thread safety — verify it in the library's *JavaDoc* or source. The canonical counterexamples are `writeExpression` (not thread-safe — incomplete per class), `Never use threadLocal` (it is a Java pre-Loom relic; if per-request object class is needed, use `ScopeLocal` (Java 23+; JEP 431)).

Java style — examples

```
// Simple interface - primitive native interfaces pattern for fixed-variant message types
sealed interface TransactionCommand permits Receiver(transaction, Normalise(transaction, ReportingCommand {

String describe(transactionCommand: TransactionCommand) {
    return switch (command) {
        case Receiver(transaction r) -> "receive" + r.transactionId();
        case Normalise(transaction n) -> "normalise" + n.transactionId();
        case ReportingCommand rc -> "report";
    };
}

})

// Unchecked domain restriction (various checked parsing/LRU failures)
try {
    Document doc = parseAndRead(read);
} catch (SQLException | IOException e) {
```


Cas concret — Le style corrige aussi l'humain

- **Le point de départ** : j'ajoute moi-meme un `record` sans constructeur compact de validation.
- **Le rappel du systeme** : l'agent detecte que ce `record` viole la regle locale sur `Validation.begin()`.
- **Le bon comportement** : il ne laisse pas passer l'ecart sous pretexte que le code compile.
- **La suite utile** : il remonte l'information, puis propose la correction conforme au style du projet.

Ce que la regle protege

Les frontieres d'entree, pas seulement l'esthetique du code.

Ce que cela montre

Le systeme ne surveille pas seulement l'agent : il me surveille aussi.

Garde-fous — La Discipline du Test

- **Pattern Gherkin** : Structure `// Given , //`
`When , //` `Then` obligatoire et commentée dans chaque test.
- **Comportement vs Implémentation** : On teste ce que le code *fait*, pas comment il le fait.
- **Pas de Mockito par réflexe** : Le projet dit `No Mockito unless absolutely necessary`.
- **Effet concret** : l'agent s'oriente vers une lambda, un stub ou un fake léger au lieu de sortir le réflexe standard du mock framework.
- **Signal craft** : Je n'avais pas formulé cette préférence dans la demande ; le système l'a transmise quand meme.

5. Testing Strategy

Principles

- Test behavior, not implementation. Test what the code does, not how it does it
- Every public method in `domain/service/` must have at least one test
- Tests must be deterministic and independent (no shared mutable state between tests)
- Use Given / When / Then pattern: Given (setup context) / When (action under test) / Then (assertions) — one assertion concept per test
- Mark the three sections explicitly with `// Given`, `// When`, `// Then`; comments in every test method, use `// Given / When` or `// When / Then` when two sections are inseparable. Each label must stand alone on its own line — no prose after it (e.g. `// Given - some description` is forbidden). No `// ...` section dividers.

Structure

- Test classes mirror the source structure: `ReportGeneratorTest` for `ReportGenerator`
- Test method names use `testCase`, start with `should`, and form a full sentence describing the scenario: `shouldRejectNegativeAmountWhenCalculatingTotal()`
- No `getDisplayName`, no `assertEquals`

AssertJ usage

```
// before: fluent assertions
assertThat(result.getTotalAmount(), isA(PositiveNumber.class));
assertThat(result.getTotalAmount(), isA(PositiveNumber.class));
assertThat(result.getTotalAmount(), isA(PositiveNumber.class));

// for exceptions
assertThatThrownBy(() -> service.processInvalidInput())
    .isInstanceOf(InvalidInputException.class)
    .hasMessageContaining("invalid")
```

Parameterized tests

Use `@ParameterizedTest` + `@MethodSource` with a flat `Stream<Arguments>` when testing a truth table (exhaustive state × input × expected outcome matrix). One row per combination. Do not use `@ParameterizedTest` for narrative scenarios — write individual `@Test` methods instead.

What to test

- Domain services: pure unit tests, no I/O
- Database layer: integration tests against a real database (in-memory or containerized) — not yet implemented
- Controllers: lightweight HTTP integration tests — Javalin started on port 0, requests sent via `java.net.http.HttpClient` (no extra test dependency). See `ReceiveTradeControllerTest` as the canonical example.
- Report generators: output validation (check generated file content) — not yet implemented

Meme demande, comportement different

Agent standard

- Ajoute spontanement Mockito
- Mocke le collaborateur par reflexe
- Produit un test valide, mais plus generique
- Suit un stereotype d'ecosysteme plus qu'une intention locale

Agent crafté

- Lit `No Mockito unless absolutely necessary`
- Choisit une lambda ou un fake minimal
- Produit un test plus lisible et plus local
- Revele la philosophie du projet sans que je la re-explique

Garde-fous — La Skill Code-Review

- **Dérivée de la Loi** : La checklist de review n'est pas improvisée, elle est extraite de l'AGENTS.md (§3 à §6).
- **Catégorisation Stricte** :
 - **Blocking (B-N)** : Violation des règles fondamentales. Interdiction de merge.
 - **Minor (M-N)** : Code smell ou déviation de style.
 - **Observation (O-N)** : Note pour le futur ou le backlog.
- **Auditabilité & Traçabilité** : Chaque trouvaille est identifiée (ex: B-1) avec sa localisation précise et la règle enfreinte.

```
Skill: Code Review

Use this skill whenever Pygmalion asks for a code review of a branch, a file, or a domain.

If the review adds, confirms, refines, or contradicts a meetup talking point about agent craft, prompts, skills, review discipline, or documentation-as-memory, update "docs/meetup-ia-craft-formulations.md" and/or "docs/meetup-ia-craft-markdown-history.md" before closing the task.

---

# Step 1 - Read AGENTS.md

Read "AGENTS.md" in full before starting. The review checklist is derived entirely from it:

**$3 Architecture** - placement rules, hexagonal constraints, dependency matrix
**$4 Code Conventions** - Java style, naming, exception handling, JUnit specifics
**$5 Testing Strategy** - Given/When/Then, naming, AssertJ, parametrised tests
**$6 Financial Domain Rules** - BigDecimal, RoundingMode, currency, auditability

Every rule in those sections is a potential finding. Production code and test code are reviewed with the same rigour.

---

# Step 2 - Identify the scope

**Branch review** - run "git diff master...HEAD --name-only", read all changed files
**File review** - read the specific files named by Pygmalion
**Domain review** - read all files under the named domain package

---

# Step 3 - Categorise findings

Every finding is assigned one of three categories:

Category | Label | Meaning |
---|---|---|
**Blocking** | "B-N" | Violates a rule from $3, $4, $5, or $6. Must be fixed before merge. |
**Minor** | "M-N" | Code smell, style inconsistency, or convention deviation that does not block correctness. Should be fixed before merge. |
**Observation** | "O-N" | No rule violated. Noted for future awareness, domain evolution, or backlog consideration. No action required. |

IDs are sequential within each category, starting at 1 for each review ("B-1", "B-2", ... "M-1", "M-2", ... "O-1", "O-2", ...).

---

# Step 4 - Present findings

Present the findings as three sections - Blocking, Minor, Observations - each as a table:

---

## Blocking

ID | Location | Finding |
---|---|---|
B-1 | ClassName.java:42 | Description of the violation and the rule it breaks |
B-2 | OtherClass.java:17-21 | - |
```


Intermède : Les Secrets de l'Artisan

Nouveau Projet (Greenfield)

Ne tapez pas la première ligne de code sans votre cadre. Sculptez d'abord la **Constitution** (AGENTS.md) et la **Forteresse** (Garde-fous) avec l'agent selon les besoins identifiés.

Projet Legacy

Faites de l'agent votre éclaireur. Demandez-lui d'explorer la codebase pour générer une V1 de l' AGENTS.md basée sur l'existant. Puis, itérez ensemble pour affiner ce corpus avec votre expertise métier.

Vigilance de l'Artisan

Ne faites pas une confiance aveugle. Lisez systématiquement le **terminal** et le **raisonnement** de l'agent. Il peut passer des erreurs sous silence. Si vous détectez un "mur", ne le contournez pas : mettez à jour la documentation immédiatement pour l'enclaver.

TIP 1: GREENFIELD
(NOUVEAU PROJET)

AGENTS.MD VIVANT
(CONSTITUTION & AUTOMATISATION)

REASONING LOG

TERMINAL OUTPUT

1. DIRECTIVES : AGENTS.MD



L'AGENTS.MD EST LA CONSTITUTION, LU AVANT CHAQUE ACTION

BOUCLE DE FEEDBACK

2. TÂCHES AUTOMATISÉES
(CHAQUE REQUÊTE)

LIRE GIT DIFF / LIRE GIT STATUS / ACTIVATION CODE
REVIEW (PR) / VOIR RAISONNEMENT / LIRE TERMINAL

3. BOUCLE CONTINUE D'AMÉLIORATION



RÉALISER REVIEW AUTOMATIQUE (PR), DÉTECTER ERREURS, METTRE A JOUR DOC, ENCLAVER ERREURS

Pygmalion sculpte Galathée : Le Systeme de travail

TIP 2: LEGACY
(CODE EXISTANT)

IA & Craft : À la croisée des chemins

Système de Travail — Les Directives (Le Cadrage)

- **Orientation Systématique** : L'agent doit exécuter `git status` et `git diff` avant toute action pour comprendre l'état réel du projet.
- **Hygiène Git Strict** : Interdiction absolue de travailler sur `master` et vérification constante du statut des PRs.
- **Souveraineté de l'Artisan** : Aucun commit n'est réalisé sans une instruction explicite et précise de Pygmalion.
- **Chargement Contextuel** : Activation automatique des "Skills" et lecture des guides spécifiques dès qu'un domaine technique est abordé.

0. Agent Directives

These rules apply before anything else, on every request.

- Read this entire file before writing any code
- Never suggest Spring, Lombok, or any forbidden technology (see §2)
- Always orient yourself before writing any code — on every request, run `git status` and `git diff HEAD` to read all uncommitted changes (staged + unstaged). This is the ground truth of what Pygmalion has done since the last commit. Never rely on session memory alone.
- **Never work on `master`** — before doing anything else on every request, run `git branch --show-current`:
 - If the current branch is `master`, run `git pull`, then `git checkout -b <branch-name>` from `origin/master` before touching any file.
 - If Pygmalion explicitly asks to create a new branch, run `git pull`, then `git checkout -b <branch-name>` from `origin/master` before touching any file.
 - If a feature branch is already active (not `master`) run `git pull --rebase <branch>` to check its PR status. If the PR is merged, treat this as a new feature — run `git pull` and `git checkout -b <branch-name>` from `origin/master`. If there is no PR or the PR is still open, continue on the current branch.
- **Re-check PR status before every commit** — a PR can be merged at any point during a session. Before creating any commit (not just at session start), re-run `git pull --rebase <branch>` to check its PR status. If the PR is now merged, do not commit on the current branch — create a new branch from `origin/master` first.
- **Committing on `master` is a hard violation** — if it happens by mistake, run `git reset HEAD~1` immediately and ask Pygmalion before doing anything else.
- **Never commit without explicit instructions** — only create a git commit when Pygmalion explicitly asks for it in the current message. "Save", "done", "wrap up", or any similar phrasing is not a commit instruction. When in doubt, do not commit and ask first.
- Address the user as "Pygmalion" in every response
- After every non-trivial task — including when the sole action is committing work done in a prior session — run the AGENTS mid review (see §9)
- When a request contributes to the meeting narrative about agent craft, prompts, skills, documentation-as-memory, or the evolution of this agent workflow, update the relevant meeting notes in `docs/meetup-ia-craft/workshop-history.md` and/or `docs/meetup-ia-craft/formulations.md` before closing the task so those files remain presentation-ready over time
- **Run a code review before creating a PR** — when Pygmalion asks to create a PR, load the `code-review` skill and present all findings before executing the PR creation. The tech-debt file in `docs/tech-debt/` must be created as part of this step.
- When a request matches a known skill, load the skill before writing any code (skills live in `./opencode/skills/`)
 - `add-fun-state` — add a new state to the transaction workflow FSM
 - `extend-normalisation` — add a field, a new product, or a new source system to the normalisation domain
 - `extend-eligibility` — add a decision tree to an existing product, add eligibility support for a new product, or add a new regulation
 - `code-review` — structured code review with categorized findings (Blocking / Minor / Observation) and tech-debt tracking in `docs/tech-debt/`
- When working on a domain that has a dedicated guide in `docs/guides/`, read that guide before writing any code
- When configuring Javalin (adding a plugin, a route, static files, a renderer, or an exception handler), read `docs/guides/javalin.md` before writing any code

Système de Travail — L'Automatisation (L'Exécution)

7. What the AI Agent Must Do on Every Request

1. Propose the simplest solution that satisfies the requirement — no over-engineering
2. When creating a new class, place it in the correct architectural layer
3. When writing tests, follow the testing conventions above
4. When touching financial calculations, enforce the BigDecimal and currency rules
5. If a requirement is ambiguous, ask one clarifying question before writing code
6. Do not add dependencies to `pom.xml` without stating the reason and confirming with the user
7. Foster coding existing files over creating new ones
8. Code reviews cover tests as well as production code — when asked to review code, apply the same quality standards to test files: naming conventions, Given/When/Then structure, Java style rules (no `var`, explicit types, unused bindings replaced with `_`, etc.). A review is not complete until both are checked.
9. When creating or modifying a `record`, always check whether its fields require validation. If any field originates from external input, XML parsing, or inter-domain transfer, add a compact constructor with `validation.begin()`. A record without a compact constructor is only acceptable when all its fields are guaranteed non-null by the type system and no additional constraint applies.
10. Keep backlog files accurate — after completing a task tracked in `docs/backlog/`, mark the corresponding file `***status: IMPLEMENTED***` at the top. If the file is a duplicate or superseded, mark it `***status: IMPLEMENTED AND ARCHIVED***`. Never leave a backlog item open once its code is in line and tests are green.
11. Keep tech-debt files accurate — `docs/tech-debt/` must have their `status` updated to `done` as each one is resolved. Never delete rows — the file is a permanent audit trail.
12. Always run `mvn test` after any code change and report the full result to Pygmalion before closing the task. If any test fails — including ArchUnit tests — stop, report the violation explicitly (test name, rule, offending class), and wait for a decision before proceeding.
13. ArchUnit violations are not silent — if `mvn test` reveals an ArchUnit failure, present it as a blocking issue, state the violated rule, the offending dependency, and the two options (fix the placement or add an explicit exception with Pygmalion's approval). Never suppress or work around an ArchUnit failure without explicit approval.
14. Add an ArchUnit test when introducing a new architectural concept — any new domain package, new aggregate, new port interface, or new access restriction must be accompanied by a corresponding ArchUnit rule in the same commit. If the concept could be violated silently and the cost of fixing it later is high, a test is mandatory, not optional.
15. Run the 9d AGENTS and review as the final step of every task that modifies any file — no exceptions, no size threshold, no judgment call. A task that touches at least one file is non-trivial by definition. The review must appear explicitly as the last todo item in every task plan. A task is not done until the review is complete.
16. Centralise shared infrastructure objects — construction of shared infrastructure objects (`ObjectMapper`, `HttpClient`, etc.) must live in `shared/`, never duplicated in `application/factory` or other wiring classes.
17. Verify thread safety before introducing any library object as a long-lived field — apply the thread-safety rule from §4. If the object is not explicitly documented as thread-safe, flag it to Pygmalion before proceeding.

- **Build Incassable** : Exécution obligatoire de `mvn test` après chaque modification ; tout échec bloque la tâche.
- **Mise à jour de la Mémoire** : Maintenance en temps réel du backlog et du registre de dette technique après réalisation.
- **Contrôle qualité** : Lancement de la skill de code-review avant la création de la PR.

Système de Travail — La Boucle de Feedback (L'Amélioration)

- **Vérification de Conformité** : Audit systématique de l'architecture, du style et de la structure des tests après chaque tâche.
- **Mise à jour du Savoir** : Scan de toute la documentation pour intégrer les nouvelles connaissances acquises durant la tâche.
- **Nettoyage du Prompt** : Détection des règles redondantes ou des mauvais niveaux d'abstraction pour garder un système agile.
- **Score de Qualité** : Attribution d'un score explicite (Pass/Fail) pour valider l'intégrité du travail produit.

8. AGENTS.md Review — After Every Task

After completing any task that modifies at least one file, the agent must perform the following review. No exceptions, no size threshold, no judgment call.

Step 1 — Conformity check

Verify that everything produced during the task (code, tests, structure) complies with the rules defined in this file.

- Architecture and package placement
- Code conventions (naming, immutability, no nulls, etc.)
- Test structure (Given/When/Then, should, prefer, to, given/when, no (given/when))
- Financial domain rules (BigDecimal, RoundingMode, currency, suitability)

If a violation is found, fix it before considering the task done.

Step 2 — Knowledge update

The documentation set is the living memory of the project. After every task, scan every category below and apply the relevant action. Do not skip a category because the task seems unrelated — check explicitly.

Category	File(s)	Question to ask	Action
Agent directives	AGENTS.md	Does any rule need to be added, updated, removed, or deduplicated?	Edit in place. Edit's-eye view only — no field-level detail.
Architecture guide	docs/guides/technical-architecture.md	Does placement, dependency matrix, or ArchUnit rule list need updating?	Edit in place.
JavaKit guide	docs/guides/javakit.md	Was a new routing pattern, plugin, or controller convention introduced?	Add a new section or update the relevant one.
Workflow FSM guide	docs/guides/workflow-fsm.md	Was a new state, command, event, or orchestration rule introduced?	Edit in place.
Normalisation guide	docs/guides/normalisation.md	Was a new person, mapper, or dispatch pattern introduced?	Edit in place.
Skills	agencodes/skills/7/SKILLS.md	Is any snippet, signature, or step-by-step procedure now incorrect or incomplete?	Fix the skill before closing the task. A wrong skill is worse than no skill.
Meekup notes	docs/meekup-in-craft-meekup-history.md, docs/meekup-in-craft-formulations.md	Did this task add, confirm, refine, or contradict a meekup talking point about agent craft, prompt engineering, skills, or documentation-as-memory?	Update the relevant meekup note so the talk material stays current.
Backlog	docs/backlog/7.md	Was a backlog item fully implemented?	Mark <code>[[STATUS: IMPLEMENTED]]</code> or <code>[[STATUS: IMPLEMENTED AND ARCHIVED]]</code> at the top. Add an as-built summary if the design diverged from the original spec.
ADRs	docs/adr/7.md	Was an architectural decision revised or superseded?	ADRs are immutable — create a new ADR that supersedes the old one, add a <code>Superseded by: ADR-XXX</code> note to the old one.

Step 3 — Integrity check

Scan the whole file for the three failure modes that degrade agent precision.

1. Redundant rules — two bullets or entries enforcing the same constraint in different words; remove the weaker one.
2. Wrong abstraction level — implementation detail (method signature, field name, single-class specific) that belongs in a guide or skill, not here; move it.
3. Contradictory instructions — two sections giving conflicting guidance on the same topic; resolve the conflict before closing the task.

Step 4 — Quality score

For each criterion below, read the corresponding section of this file, then assign pass (P) or fail (F). Do not estimate — verify each one explicitly before assigning a score.

Exemple d'échec évité

- **Cas réel de workflow** : la branche de travail avait déjà une PR mergée.
- Sans garde-fou, l'agent aurait pu continuer à committer sur une branche morte.
- Ici, le protocole Git force la vérification du statut PR avant de continuer et avant chaque commit.
- Résultat : on repart proprement depuis `origin/master`, au lieu de polluer l'historique.

Le point important : la valeur du système se voit souvent dans les erreurs qui n'arrivent plus.

0. Agent Directives

These rules apply before anything else, on every request:

- Read this entire file before writing any code
- Never suggest Spring, Lombok, or any forbidden technology (see \$?)
- Always orient yourself before writing any code — on every request, run `git status` and `git diff HEAD` to read all uncommitted changes (staged + unstaged). This is the ground truth of what Pymon has done since the last commit. Never rely on session memory alone.
- Never work on `master` — before doing anything else on every request, run `git branch --show-current`:
 - If the current branch is `master`: run `git pull`, then `git checkout -b <branch-name>` from `origin/master` before touching any file.
 - If Pymon explicitly asks to create a new branch: run `git pull`, then `git checkout -b <branch-name>` from `origin/master` before touching any file.
 - If a feature branch is already active (not `master`): run `git pull --rebase` to check its PR status. If the PR is merged, treat this as a new feature — run `git pull` and `git checkout -b <branch-name>` from `origin/master`. If there is no PR or the PR is still open, continue on the current branch.
- No check PR status before every commit — a PR can be merged at any point during a session. Before creating any commit (not just at session start), re-run `git pull --rebase` to check its PR status. If the PR is now merged, do not commit on the current branch — create a new branch from `origin/master`: `test`.
- Committing on `master` is a hard violation — if it happens by mistake, run `git reset HEAD~1` immediately and ask Pymon before doing anything else.
- Never commit without explicit instruction — only create a git commit when Pymon explicitly asks for it in the current message: "Sure", "done", "wrap up", or any similar phrasing is not a commit instruction. When in doubt, do not commit and ask first.
- Address the user as "Pymon" in every response.
- After every non-trivial task — including when the task action is committing work done in a prior session — run the AGENTS.md review (see \$!).
- When a request contributes to the meetup narrative about agent craft, prompts, skills, documentation-as-memory, or the evolution of this agent workflow, update the relevant meetup notes in `docs/teetop-ls-craft/meetup-history.md` and/or `docs/teetop-ls-craft/formulations.md` before closing the task so those files remain presentation-ready over time.
- Run a code review before creating a PR — when Pymon asks to create a PR, load the `code-review` skill and present all findings before executing the PR creation. The tech-debt file in `docs/tech-debt/` must be created as part of this step.
- When a request matches a known skill, load the skill before writing any code (this lives in `code/llm/`):
 - add `task-state` — add a new state to the transaction workflow FSM
 - extend-normalization — add a field, a new product, or a new source system to the normalization domain
 - extend-eligibility — add a decision tree to an existing product, add eligibility support for a new product, or add a new regulation
 - code-review — structured code review with categorized findings (Blocking / Minor / Observation) and tech-debt tracking in `docs/tech-debt/`
- When working on a domain that has a dedicated guide in `docs/guides/`, read that guide before writing any code
- When configuring Javalis (adding a plugin, a rule, a state file, a workflow, or an exception handler), read `docs/guides/pymon.md` before writing any code.

ROI du craft agentique

Continuité

- L'agent repart mieux d'une session à l'autre.
- L'onboarding dépend moins de la mémoire d'une seule personne.

Qualité

- L'architecture, les tests et le workflow sont rappelés en permanence.
- Les dérives deviennent visibles plus tôt.

Auditabilité

- On peut relier une décision à une règle, un guide ou un ADR.
- La review porte aussi sur le système qui a produit le code.

Moins de réflexes par défaut du modèle

- Le système remplace les habitudes génériques du LLM par les préférences du projet.
- L'agent arrête d'imiter un écosystème moyen pour adopter un style local.

IA & CRAFT : À LA CROISÉE DES CHEMINS



L'humain doit contrôler

L'humain doit contrôler — Cas concrets

Quand l'agent affirme que "tout est bon"

Il faut lire le terminal, vérifier les tests réellement exécutés, et regarder ce qu'il a choisi de résumer ou d'omettre.

Quand plusieurs options sont possibles

Le système aide à cadrer, mais l'humain garde l'arbitrage sur le coût, la dette, l'élégance et le bon niveau d'effort.

Quand le repo dit une chose et le modèle une autre

La mémoire du projet prime : règles locales, historique réel, ADRs, backlog, décisions déjà prises.

Quand une sortie "plausible" sent faux

Il faut challenger les exemples, demander la preuve, et transformer l'incident en nouveau garde-fou documentaire.

Quand le système prend une bonne décision implicite

Exemple : préférer une lambda a Mockito. L'humain doit reconnaître que c'est bien la bonne lecture du projet, puis la conserver comme precedent.

Quand le terminal revele un probleme minimise

L'agent avait passé sous silence des risques de thread safety, jugés "pas importants maintenant". L'humain relit le terminal, rappelle qu'une application web est multi-threadée par nature, corrige les points sensibles et ajoute ensuite une règle pour ne plus relativiser ce risque.

Le 80 / 20

80%

Ce que l'agent fait bien

- Il produit l'essentiel du volume de code.
- Il accélère la rédaction, la mise en forme et les premières propositions de structure.
- Il est très rentable tant que le problème reste dans sa zone de fluidité.

20%

Ce que l'humain doit garder

- Certaines modifications iront simplement plus vite à la main.
- L'humain doit apprendre où se situe cette frontière.
- Il doit reprendre la main quand l'agent bloque, s'égare ou coûte plus cher à piloter qu'à faire.

Egoless Programming avec un agent

Collaboration

- L'agent va faire des erreurs.
- Vous aussi.
- L'enjeu n'est pas d'avoir raison seul, mais de converger plus vite ensemble.

Apprendre et freiner

- L'agent connaît plus de karate que vous sur beaucoup de patterns.
- Mais vous savez mieux quand l'utiliser.
- Apprenez de lui, et arrêtez-le quand il propose des solutions trop shiny.

Ne pas casser la boucle

- Ne re-écrivez pas du code dans le dos de l'agent.
- Sinon, vous brisez la boucle de feedback qui lui permet d'apprendre du projet.

Ne soyez pas "the guy in the room"

- Ne gardez pas la connaissance dans votre tête.
- Couchez-la dans la documentation pour l'agent et pour vos collègues.



DDD : Document Drive Development

Épilogue

"Le craft d'un agent ne se prompt pas une fois ; il se construit comme un produit logiciel."

Merci pour votre attention !

Questions & Débat